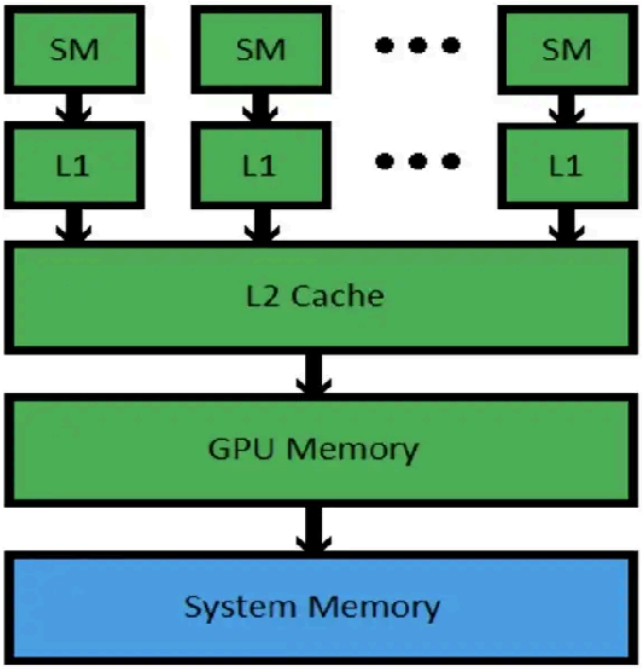
GPU相关知识

GPU-CPU机器的存储层次和性能

以下是一个较为“通用”的CPU-GPU存储层次图示



知乎 @sudit

存储层级	技术描述	容量（单卡）	峰值带宽（单卡）	主要用途
PCIe 接口 (主机至GPU)	PCI Express Gen3 总线接口	取决于内存容量（我们是30G）	**单向16GB/s 双向32 GB/s**	连接CPU和GPU，负责在**主机内存（Host Memory）和GPU显存（HBM2）**之间传输数据（如初始输入数据和最终结果）。
HBM2 显存	高带宽内存 (High Bandwidth Memory)	16GB 或 32GB	**900 GB/s**	存储模型参数、梯度、激活值、大规模数据集等，是GPU的主内存。
L2 Cache (二级缓存)	所有SM共享的片上缓存	约 6144KB	-	减少对HBM2显存的访问延迟，提高数据复用效率。
SMEM (Shared Memory)	每个流多处理器(SM)的专用低延迟内存	128KB/SM	-	可由程序员显式管理，用于线程块(Thread Block)内的线程协作和数据共享，或作为缓存。
寄存器 (Registers)	每个SM核心私有的最快存储单元	每个SM约256KB	-	存储线程的私有变量和中间计算结果，速度极快。

我们的项目面临瓶颈

查询计算并不复杂，是memory-bound。在各个存储层次中，内存到HBM的传输带宽（即PCIE）是瓶颈。所以，我们不过多优化核函数，而是着重优化将向量从内存上传到GPU的过程。

可能的优化点

1. 尽量让数据留在GPU显存中，涉及算法避免将数据进行换入和换出
2. 多使用DMA异步传输，隐藏传输延迟

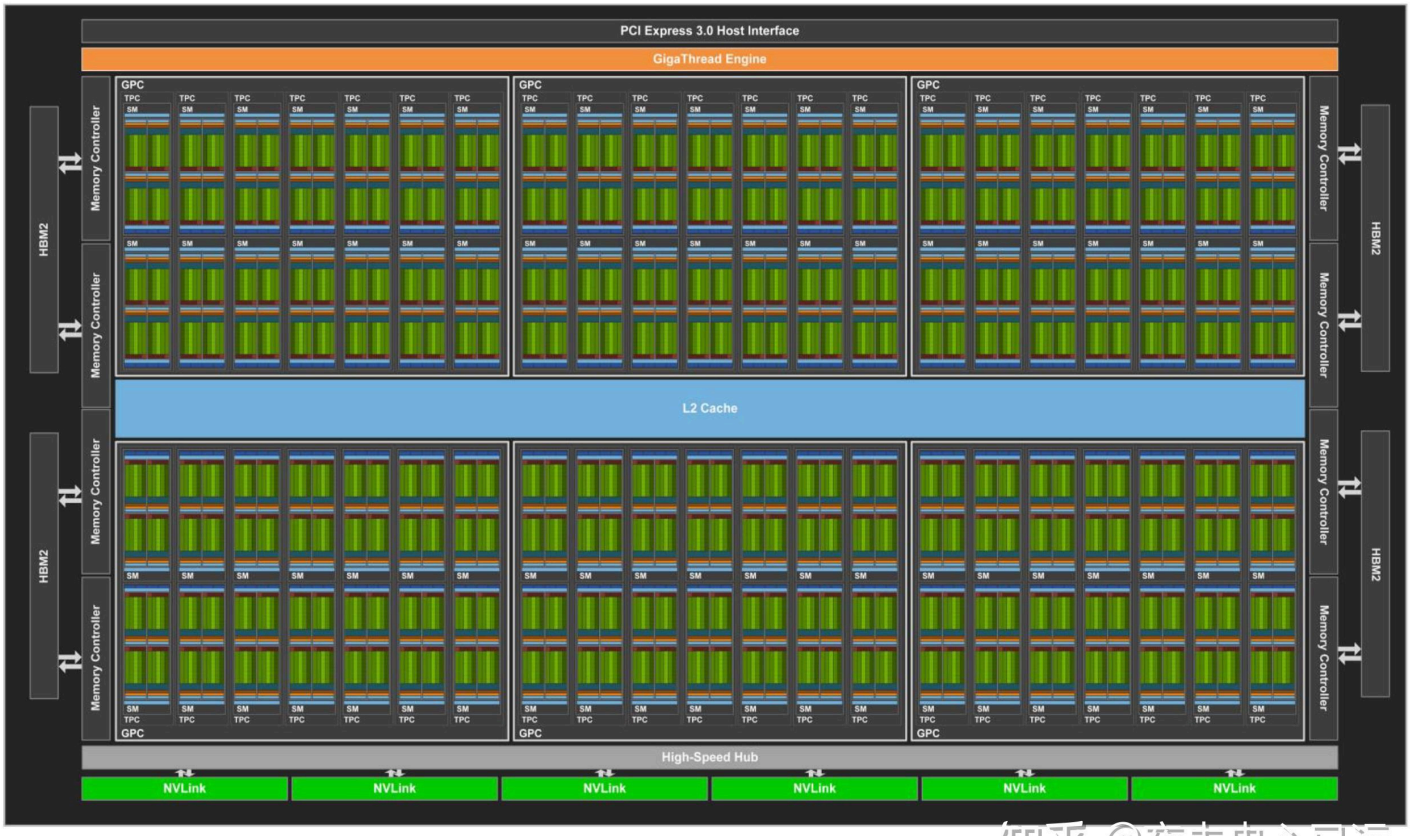
3. 使用锁页内存(Pinned Memory)增加传输速度

我们的计算资源：单张V100 GPU

参考：https://zhuanlan.zhihu.com/p/190595859739492766 8

全称Tesla V100，Volta架构

V100整体结构



知乎 @夜未央心已远

Figure 4. Volta GV100 Full GPU with 84 SM Units

性能参数

- Tensor Core 混合精度加速：内置 640 个 Tensor Core，支持 FP16 输入/FP32 累加的 4x4 矩阵乘加运算（**专门为加速矩阵乘法设计，和CuBLAS库应该是专门适配的**），提供 125 TFLOPS 混合精度算力，深度学习训练性能较前代提升 12 倍。
- 流式多处理器（SM）重构：每个 SM 包含 64 FP32、64 INT32、32 FP64 核心及 8 个 Tensor Core，支持整型与浮点运算并行，L1 缓存/共享内存动态分配（最高 128 KB/SM），显存带宽利用率达 95%。
- HBM2 显存与高带宽设计：采用 4 堆叠 HBM2 显存，4096 位总线实现 900–1134 GB/s 带宽（较前代提升 23%），32 GB 容量支持 ECC 纠错，延迟降低 40%。
- NVLink 2.0 多 GPU 互联：6 通道 NVLink 2.0 互联，总带宽 300 GB/s（单链路 25 GB/s），8 卡集群扩展效率超 90%，支持 IBM Power 平台缓存一致性。
- 独立线程调度机制 ：突破传统 SIMT 模型，实现线程级细粒度同步，支持动态分支与原子操作，非规则算法（如粒子模拟）性能提升 70%。
- 能效优化与功耗管理：台积电 12nm FFN 工艺，300W TDP 下提供 15.7 TFLOPS FP32 算力，50% 功耗限制下仍保持 85% 性能，单位算力能效提升 50%。

V100 SM内部结构

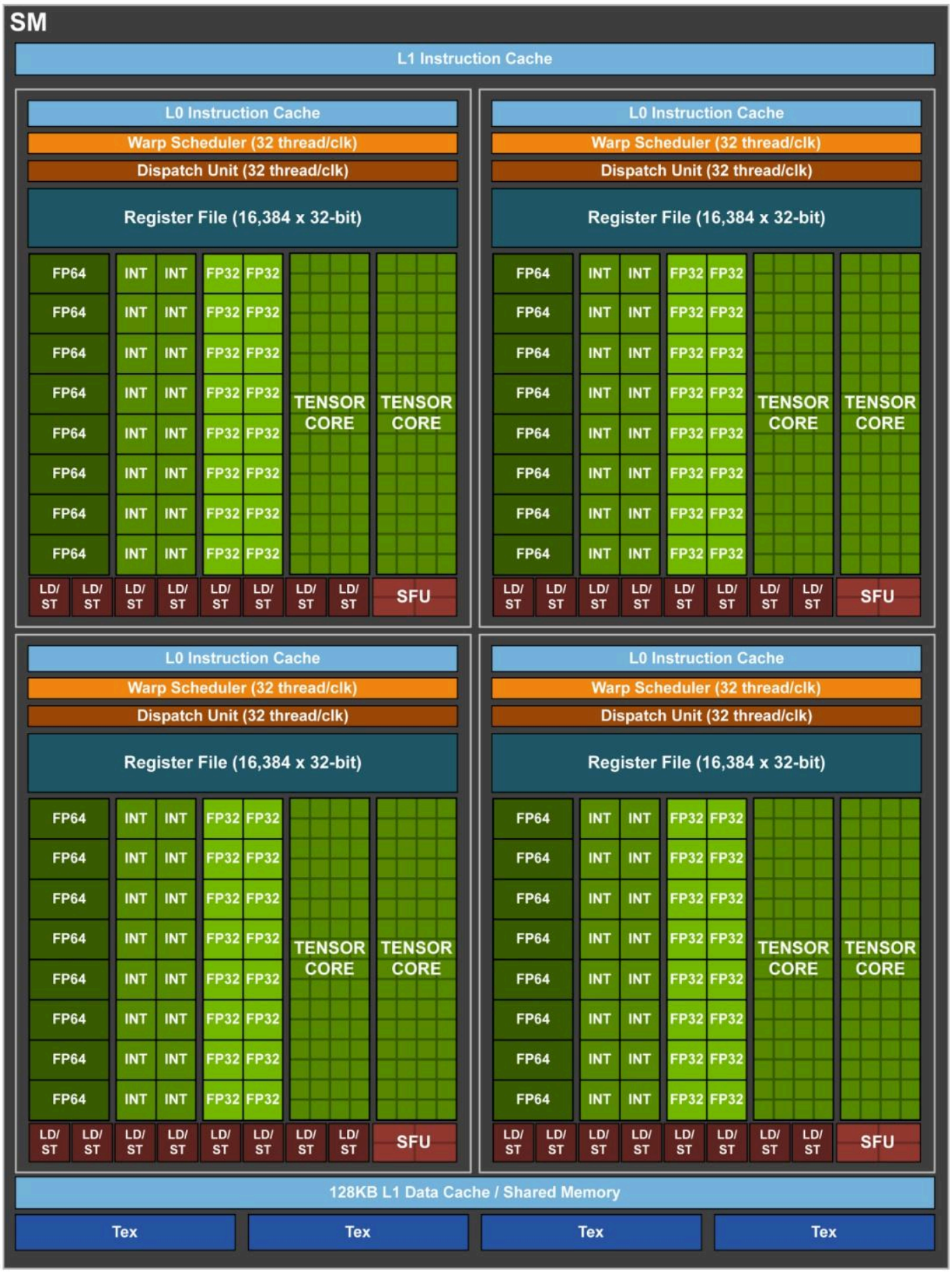


Figure 5. Volta GV100 Streaming Multiprocessor (SM)

知乎 @夜未央心已远

L1缓存可以编程，未被分配到共享内存的可以作为L1缓存使用

GPU Warp 线程束

Warp 是NVIDIA GPU架构中的基本执行单位：

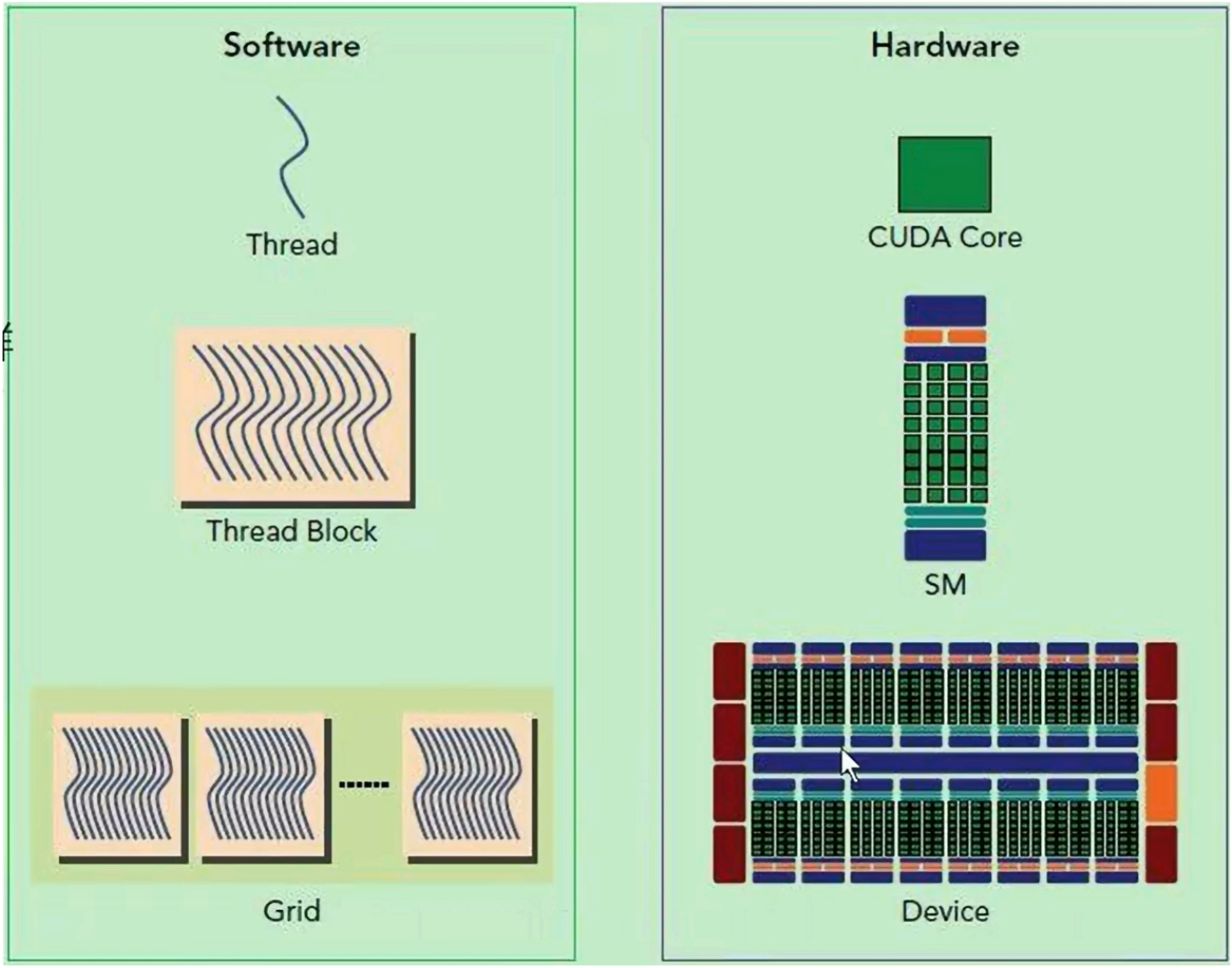
- 一个Warp包含 **32个线程**
- SIMD 单指令多数据
- 是GPU硬件级别的并行单元

CUDA编程相关支持

GPU上生成随机数：CuRand

CUDA线程模型

thread是实际存在的线程，block和grid都是逻辑抽象



cuda+pipeline

我们使用大量生成的随机数据，初步搭建了数据pipeline。对 1. **L2范数/Normalizer** 2. **矩阵乘法** 3. **余弦距离** 三个算子进行了初步测试。其中，**L2范数/Normalizer**测试了多批次数据的流水线，另外两个算子只测试了单批次。

每个cuda算子包括两部分：

1. cuda和函数，当数据传递到GPU上时应该怎样计算
2. cuda-c的wrapper，供pgvector系统进行调用，负责处理如何将数据传输到GPU并调用核函数

余弦距离（余弦相似度）

```
=== Test1: 基本余弦距离测试 ===
测试向量组大小: 1024 个查询向量 × 1024 个数据向量
向量维度: 1024
内存使用量: 12 MB
[CPU Timer] GPU Memory Allocation took: 3 ms
[CUDA Event Timer] H2D Data Transfer took: 2.67136 ms
[CUDA Event Timer] Kernel Execution took: 108.585 ms
[CUDA Event Timer] D2H Data Transfer took: 5.36835 ms
[CPU Timer] GPU Memory Free took: 1 ms
result
0
GPU耗时: 453 ms
CPU耗时: 1544 ms
加速比: 3.40839x
基本余弦距离测试完成 ✓
```

主要原因是除以L2 Norm的时候，本来n^2个线程并行变成了n个线程并行，而且只开了一个cuda流

另外，两个向量的余弦相似度等价于normalize

L2 Norm

这里是把L2 Norm算子放进normalizer来测试了

```
=== 大规模数据压力测试 ===
测试规模: 1024 lists × 1024 vectors × 1024 dimensions
总内存使用量: 4096 MB
生成大规模数据: 1024 lists, 1024 vectors per list, 1024 dimensions
已生成 0/1024 lists
已生成 100/1024 lists
已生成 200/1024 lists
已生成 300/1024 lists
已生成 400/1024 lists
已生成 500/1024 lists
已生成 600/1024 lists
已生成 700/1024 lists
已生成 800/1024 lists
已生成 900/1024 lists
已生成 1000/1024 lists
大规模数据生成完成 ✓
注册内存为页锁定...
内存注册完成 ✓
[CPU Timer] Create cuda streams and cuda events took: 0 ms
[CPU Timer] Async GPU Memory Allocation took: 43 ms
[CUDA Event Timer] Pipeline Process took: 230.42 ms
[CPU Timer] Synchronize Streams took: 453 ms
[CPU Timer] Async Resource Free took: 0 ms

=== 性能统计 ===
总处理时间: 729 ms
平均每list处理时间: 0.711914 ms
向量处理吞吐量: 1.43838e+06 vectors/second
内存带宽: 10.9739 GB/s
大规模压力测试完成 ✓
all test passed!
```

自己写，已经完成并测试

矩阵乘法

```
=== Test1: 基本矩阵乘法测试 ===
测试矩阵大小: 10x1024 × 1024x1024
内存使用量: 4 MB
[CPU Timer] GPU Memory Allocation took: 314 ms
[CUDA Event Timer] H2D Data Transfer took: 1.23213 ms
[CUDA Event Timer] Kernel Execution took: 6.8287 ms
[CUDA Event Timer] D2H Data Transfer took: 0.084704 ms
[CPU Timer] GPU Memory Free took: 0 ms
result
0
GPU耗时: 324 ms
CPU耗时: 14 ms
加速比: 0.0432099x
GPU性能: 0.0647269 GFLOPS
all_test_passed
```

用cuBLAS库的GEMM接口，

```
cublasStatus_t cublasSgemm(cublasHandle_t handle,
                           cublasOperation_t transa,     # 矩阵a是否转置
                           cublasOperation_t transb,     # 矩阵b是否转置
                           int m, int n, int k,
                           const float *alpha,
                           const float *A, int lda,
                           const float *B, int ldb,
                           const float *beta,
                           float *C, int ldc);
```

参数

- **handle**：cuBLAS 库的上下文句柄，用于管理 GPU 计算状态
- **transa**, **transb** 矩阵a,/b是否转置
- **m**,**n**,**k**: 矩阵A * 矩阵B = 矩阵C [m,k] * [k*n] -> [m,n]
- **alpha**和**beta**控制矩阵乘加运算，alpha=1.0 beta=0.0表示纯矩阵乘法
- **d_A**, **d_B**, **d_C** 三个矩阵的设备地址
- **lda**, **ldb**, **ldc**: 主维度，列优先存储是每列的元素数量
  cuBLAS 默认按列优先存储，而 C/C++ 程序通常按行优先存储数据

数值精度

但是精度和CPU有差别（也不知道哪个是正确的）
4e-5的概率误差为1%
4e-4的概率误差为0.1%
4e-3的概率误差为0.001%
似乎还可以接受，先用着

一些发现

1. GPU冷启动时在malloc等操作上开销极大，需要在数据库启动前自动完成冷启动，
2. GPU计算浮点数和CPU有误差，可能影响recall

可能的优化点

1. 用float4数据类型（核函数）
2. 改进线程块尺寸（核函数）
3. 将任务切分成块，多占显存（核函数）
4. 开多个cuda流（wrapper）

NVIDIA 可视化工具

nsight

内存监测工具

gperftools 没跑成功
Valgrind 跑成功了，但是不能从内存问题自动定位到函数和行数